

2.pdf



University of Engineering & Management

Document Details

Submission ID

trn:oid::3618:103124702

Submission Date

Jul 1, 2025, 2:33 PM GMT+5:30

Download Date

Jul 5, 2025, 12:50 AM GMT+5:30

File Name

2.pdf

File Size

410.7 KB

8 Pages**5,856 Words****39,063 Characters**

100% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **1** Not Cited or Quoted 100%
Matches with neither in-text citation nor quotation marks
-  **0** Missing Quotations 0%
Matches that are still very similar to source material
-  **0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 17%  Internet sources
- 12%  Publications
- 100%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

-  **1** Not Cited or Quoted 100%
Matches with neither in-text citation nor quotation marks
-  **0** Missing Quotations 0%
Matches that are still very similar to source material
-  **0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 17%  Internet sources
- 12%  Publications
- 100%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

-  **1** Submitted works

New Mexico State University on 2025-06-16

100%

Kubernetes Scaling: A Comprehensive Review of Scalability in Kubernetes

Wijesekera D. T. S.

Faculty Of Graduate Studies &
Research

Sri Lanka Institute of Information
Technology

Malabe, Sri Lanka

ms23464582@my.sliit.lk

Dr. Wijendra D. R.

Faculty Of Graduate Studies &
Research

Sri Lanka Institute of Information
Technology

Malabe, Sri Lanka

dinuka.w@sliit.lk

Abstract - Kubernetes has become the leading container orchestration platform due to its powerful scalability features, enabling dynamic resource management and efficient workload handling in cloud-native environments. This review examines Kubernetes scaling mechanisms at both the application and cluster levels, focusing on Horizontal Pod Autoscaler (HPA), Vertical Pod Autoscaler (VPA), and event-driven scaling with KEDA for adaptive application scaling. At the cluster level, Cluster Autoscaler (CA), Karpenter, Cluster Proportional Autoscaler (CPA), and Cluster Proportional Vertical Autoscaler (CPVA) optimize node provisioning and resource allocation. Despite these advancements, challenges persist, including reactive scaling delays, resource fragmentation, inconsistent scaling decisions across multiple autoscalers, and security vulnerabilities like Economic Denial of Sustainability (EDoS) attacks. To address these issues, emerging trends in AI-driven observability, predictive analytics, and unified autoscaling frameworks offer proactive scaling, anomaly detection, and self-healing capabilities. This review synthesizes academic research and industry practices to highlight the current state, challenges, and future directions of Kubernetes scalability, emphasizing the need for intelligent, adaptive, and secure scaling solutions.

Keywords – Kubernetes, Scalability, Autoscaler, KEDA, Karpenter

I. INTRODUCTION

The market for application containers is rapidly expanding, with forecasts predicting growth to USD 19.41 billion by 2029 at a CAGR of 28.89% [1]. This surge reflects the transformative impact of containerization in modern software technology, where it has revolutionized the development and deployment of applications. As driven by the modern-day requirement in IT for high scalability, flexibility, and automation in application environments, containers are an essential entity in today's software development and deployment lifecycles. This makes it easy for organizations to cater for the demand for fast and efficient delivery of software. One of the major enablers of this change is microservices, which is a style of designing an application as a collection of loosely coupled components that can be independently deployable, developable, and scalable [2]. Microservices employ the concept of modularity as well as isolation of components, and since containers implement these concepts, it makes easy for the new cloud-based applications to employ the use of containers.

Managing and orchestrating containers on a scale presents substantial operational challenges, particularly in environments that require automated scaling, optimized resource allocation, and robust fault tolerance [3] [4]. As the adoption of containers continues to grow, Kubernetes has emerged as the de facto standard for container orchestration, enabling organizations to efficiently manage, scale, and deploy containerized applications. This is evidenced by its

status as the second largest open-source project globally, following Linux, and its role as the primary container orchestration tool for 71% of Fortune 100 companies [5].

Kubernetes offers numerous benefits for modern infrastructure, and one of its most significant advantages is scalability. Unlike traditional virtualization methods, which often require manual intervention to provision new virtual machines (VMs), allocate additional resources, and configure networking, Kubernetes introduces a dynamic and automated approach to scaling [6]. Designed to handle dynamic workloads, Kubernetes provides multiple scaling mechanisms at the application level, including Horizontal Pod Autoscaler (HPA), Vertical Pod Autoscaler (VPA), which adjust pod replicas and resource allocations based on real-time demand [7]. At the cluster level, Kubernetes utilizes solutions like Cluster Autoscaler and Karpenter to dynamically manage the number of nodes, ensuring efficient resource allocation and cost optimization [8].

Horizontal scaling also referred to as "scaling out" addresses increased application demand by adding more pods thereby distributing workload across multiple parallel resources [6]. At the pod level, this approach often leverages the Horizontal Pod Autoscaler (HPA), which monitors CPU, memory, or custom-defined metrics to determine when additional replicas should be spun up [9]. By creating new pod instances, horizontal scaling boosts the overall throughput while preserving the stateless nature of microservices-based applications. Vertical Scaling in Kubernetes is commonly implemented via the Vertical Pod Autoscaler (VPA) focuses on adjusting resource requests (e.g., CPU and memory) for containers as workload demands evolve [7] [10]. This ensures that integrating vertical scaling in Kubernetes environments can significantly reduce infrastructure overhead by minimizing both overprovisioning and resource bottlenecks [11].

Kubernetes scaling can be enhanced by combining traditional vertical scaling with event-driven and proportional approaches. The Vertical Pod Autoscaler (VPA) dynamically tunes per-pod CPU and memory allocation to reduce over- or under-provisioning, while Kubernetes Event-Driven Autoscaling (KEDA) scales workloads in response to external metrics such as queue lengths or message stream activity [12]. In addition, emerging techniques like the Cluster Proportional Autoscaler and Cluster Proportional Vertical Autoscaler go beyond a purely pod-centric view, adjusting resource requests proportionally to overall cluster capacity [13] [14].

Node scaling in Kubernetes extends beyond simply adjusting pod resource requests or replica counts, focusing instead on adding or removing entire worker nodes to match workload demands. Often achieved through the Cluster Autoscaler (CA), this approach continuously monitors

1 unschedulable pods and provisions new nodes (or removes underutilized ones) in response [15]. Modern node-level solutions such as Karpenter provide additional optimizations like instance-type flexibility, faster spin-up times, and real-time selection of the most cost-effective or performance-appropriate VM flavors [16].

Building on these diverse autoscaling strategies ranging from horizontal and vertical pod adjustments to event-driven triggers and node-level provisions, Kubernetes delivers a multifaceted approach to maintaining application responsiveness and operational efficiency. Nevertheless, challenges remain in reconciling these techniques on a scale, particularly in hybrid or resource-constrained environments where cost, latency, and performance requirements must be carefully balanced.

II. REVIEW OF THE LITERATURE

A. Background of Kubernetes

In 1999, FreeBSD introduced Jails, an OS-level virtualization mechanism designed primarily for security-focused isolation [17]. FreeBSD Jails allow applications or entire software stacks to operate in isolated environments with their own processes, filesystems, and user spaces, while still utilizing the same underlying host operating system kernel [17]. In a similar manner, Solaris Zones introduced by Daniel Price and Andrew Tucker in paper 'Solaris Zones: Operating System Support for Consolidating Commercial Workload', gave a technique to build isolated operating spaces or instances of the Solaris OS [18]. It has been initiated by Sun Microsystems in 2004 with Solaris 10 [19].

The growing adoption of Docker in the early 2010s led to an increasing demand for container orchestration tools capable of managing large-scale containerized workloads across multiple hosts. To handle resource management at an unprecedented scale, Google developed Borg, its first unified container management system, capable of efficiently scheduling and managing billions of containers in production [20]. According to Verma et al. (2015), Borg is a highly scalable and fault-tolerant cluster management system that employs a two-level scheduling architecture to efficiently allocate resources across thousands of machines, leveraging priority-based scheduling, resource overcommitment, and task preemption to maximize utilization and ensure workload reliability in Google's data centers [21]. Alongside Borg, Google developed Omega, a successor designed to overcome some of Borg's limitations. As described by Schwarzkopf et al. (2013), Omega introduced a more flexible architecture, incorporating optimistic concurrency control, which enabled parallel scheduling of operations, enhancing scalability in highly dynamic, multi-tenant environments [22].

Building upon the core principles of Borg, Google launched Kubernetes (K8s) in 2014 as an open-source container orchestration platform, leveraging community-driven contributions to enhance scalability and automation in cloud-native environments [23]. Since its inception, Kubernetes has continuously evolved, introducing new features, performance optimizations, and enhanced security mechanisms to meet the growing demands of modern cloud infrastructure.

B. Overview of Kubernetes Architecture

Kubernetes is designed as a modular and extensible system that orchestrates the deployment, scaling, and management of containerized applications [3] [11]. It follows a master-worker architecture, where the control plane is responsible for managing cluster state and the worker nodes execute workloads [11]. This section provides an in-depth analysis of the key components of Kubernetes architecture.

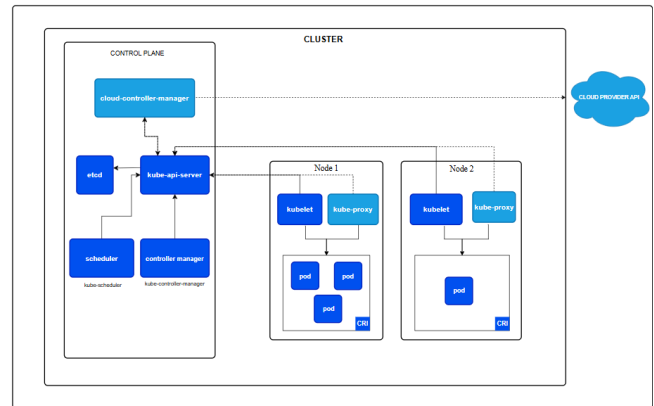


Figure 1: Kubernetes Architecture [24]

1) Kubernetes Control Plane

The control plane is the central management layer that maintains the desired state of the cluster.

API Server: Acts as the front-end interface for the cluster, handling communication between internal components and external users.

Controller Manager: Maintains cluster stability by continuously monitoring the state of Kubernetes objects and reconciling deviations from the desired state. It includes controllers for replication, endpoint updates, and node lifecycle management.

Scheduler: Assigns workloads (Pods) to available worker nodes based on resource requirements, affinity rules, and constraints.

etcd: A distributed key-value store that serves as Kubernetes' primary database, storing cluster state and configuration data.

Cloud Controller Manager: Enables Kubernetes to integrate with cloud providers by abstracting infrastructure-specific operations, such as load balancer provisioning, node lifecycle management, and persistent volume integration. It runs separately from the core Kubernetes controllers, ensuring cloud provider dependencies do not affect cluster performance.

2) Worker Nodes and Runtime Environment

Worker nodes in Kubernetes execute containerized workloads while maintaining network connectivity and efficient resource management. The runtime environment consists of multiple key components that ensure smooth pod execution, lifecycle management, and inter-service communication.

Kubelet - Node Agent: The primary agent on each worker node responsible for registering the node with the API server, enforcing pod specifications, and ensuring containers are running as expected. It communicates directly with the container runtime to manage workload execution.



Kube-proxy: Maintains virtual IP-based networking for Kubernetes services and implements NAT rules for inter-pod communication.

Container Runtime: Responsible for pulling images, launching containers, and managing their lifecycle. Kubernetes supports multiple runtimes via the Container Runtime Interface (CRI)

C. Scaling Concepts and Mechanisms

Kubernetes enables automated resource management by dynamically scaling workloads and infrastructure based on real-time demand. Autoscaling mechanisms in Kubernetes operate at different levels:

1) Pod-Level Autoscaling

In Kubernetes, the smallest deployable unit is the Pod [25]. It serves as the fundamental building blocks for deploying workloads, making pod-level autoscaling the foundation of Kubernetes scalability.

a) Horizontal Scaling

Horizontal scaling, also known as scaling out, is the process of increasing or decreasing the number of replica instances of a workload to accommodate changing resource demands. In Kubernetes, this is achieved by dynamically adjusting the number of Pods [26]. Kubernetes provides a built-in autoscaling mechanism, the Horizontal Pod Autoscaler (HPA), which automatically adjusts the number of pod replicas in a Deployment, StatefulSet, or ReplicaSet based on real-time metrics [26].

The Kubernetes API is the core component of Kubernetes that exposes its functionality through a RESTful interface [27]. HPA is under the autoscaling API group [9]. Kubernetes' Horizontal Pod Autoscaler (HPA) has evolved through different API versions, with HPA v2 introducing several improvements over HPA v1. While HPA v1 provided a basic framework for CPU-based autoscaling, HPA v2 extends metric support, improves scaling control [28] [29].

Horizontal scaling in Kubernetes follows a control loop mechanism, continuously monitoring workload demands and adjusting the number of pod replicas accordingly [11] [30]. First Kubernetes collects resource utilization metrics through the Metrics API (e.g., CPU and memory usage from kube-metrics-server) or custom metric providers like Prometheus [30]. The default sync period for HPA is 15 seconds, meaning it evaluates metrics at this interval [11]. Once the required metrics are gathered, Kubernetes determines the optimal number of pod replicas using a predefined autoscaling formula [9].

$$desiredReplicas = \left\lceil currentReplicas \times \frac{currentMetricValue}{desiredMetricValue} \right\rceil$$

Figure 2: HPA autoscaling formula

Example Scenario: CPU-Based Scaling with max replicas count as 4

Consider a Deployment with:

- **Current number of pods: 2**
- **Current CPU usage per pod: 150m**
- **Target CPU utilization: 60m**

Using the formula:

$$desiredReplicas = \left\lceil 2 \times \frac{150}{60} \right\rceil = \left\lceil 5 \right\rceil = 5$$

Figure 3: HPA autoscaling formula example 1

Even though the calculated desiredReplicas is 5, Kubernetes will only scale up to the maximum limit of 4 replicas

Example Scenario: Scaling Down

If CPU utilization drops to 40m:

$$desiredReplicas = \left\lceil 4 \times \frac{40}{60} \right\rceil = \left\lceil 4 \times 0.67 \right\rceil = \left\lceil 2.67 \right\rceil = 3$$

Figure 4: HPA autoscaling formula example 2

HPA will reduce the pod count from 4 to 3, but stabilization policies may delay this action to prevent rapid downscaling.

Once the scaling decision is made, the Kubernetes control plane executes the necessary adjustments. If scaling up, the Deployment, StatefulSet, or ReplicaSet controlling the application is updated with an increased replica count, and the Kubernetes Scheduler provisions new pods across the available worker nodes. Conversely, if scaling down, excess pods are gradually terminated to free up cluster resources while ensuring minimal disruption to active workloads. This automated approach enables Kubernetes to dynamically match application demand with available infrastructure capacity, thereby optimizing cost and performance.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

Figure 5: Example YAML manifest of an HPA.

b) Vertical Pod Autoscaler

Vertical scaling, also referred to as scaling up or down, is the process of dynamically adjusting the resource allocation. In Kubernetes, vertical scaling is implemented through the Vertical Pod Autoscaler (VPA), which continuously monitors pod resource usage and adjusts CPU and memory requests dynamically to match application needs [7]. However, unlike Horizontal Pod Autoscaler (HPA), which is natively integrated with Kubernetes, VPA is not included by default in Kubernetes distributions. It is maintained as a separate

1

component by the Kubernetes community and must be installed and configured independently [7]. VPA relies on the Metrics Server to collect usage data, making the Metrics Server a prerequisite for accurate scaling decisions [7].

The Vertical Pod Autoscaler (VPA) in Kubernetes consists of three components: the Recommender, which analyzes resource usage to suggest optimal CPU and memory requests; the Updater, which applies these recommendations by evicting and recreating pods for improved performance; and the Admission Controller, which adjusts resource requests during pod creation to ensure accurate configurations [10].

The Vertical Pod Autoscaler (VPA) has three operational modes: Off Mode, which only suggests resource adjustments without applying them, ideal for testing and manual oversight; Auto Mode, which automatically adjusts resource requests based on real-time usage but may cause temporary disruptions due to pod restarts; and Initial Mode, which sets optimal resource requests during pod creation without affecting running pods, ensuring consistent startup configurations [10].

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: php-apache-vpa
  namespace: default
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  updatePolicy:
    updateMode: "Auto"
  resourcePolicy:
    containerPolicies:
      - containerName: "*"
        minAllowed:
          cpu: "100m"
          memory: "256Mi"
        maxAllowed:
          cpu: "2"
          memory: "4Gi"
        controlledValues: "RequestsOnly"
```

Figure 6: Example YAML manifest of a VPA

c) Event-driven Scaling

Event-driven scaling is a reactive autoscaling approach that adjusts the number of pods in response to external event sources such as message queues or custom metrics [7]. In Kubernetes, this is effectively managed by the Kubernetes-based Event Driven Autoscaler (KEDA), a lightweight and single-purpose component designed to enable event-driven scaling of containers within Kubernetes clusters [31]. Traditional autoscaling methods like HPA rely on static thresholds and react to changes after they occur, leading to inefficiency during sudden traffic spikes or demand drops [32]. KEDA addresses these limitations by enabling event-driven scaling that triggers scaling actions in response to real-time events, allowing for more immediate and context-aware decisions. The integration of machine learning with KEDA enhances this further by enabling predictive autoscaling, which analyzes historical data to forecast future workload demands [32].

KEDA enhances scalability by supporting over 50 event sources, including Kafka, RabbitMQ, AWS SQS, Azure Monitor, Prometheus, Redis, MongoDB, and GCP Pub/Sub [31]. To achieve this, KEDA introduces several Custom Resource Definitions (CRDs), which include **cloudeventsources.eventing**, **lustertriggerauthentications**, **scaledjobs**, **scaledobjects**, **triggerauthentications** [33]. These CRDs define the event sources and scaling behavior, allowing Kubernetes to respond dynamically to external events. In addition to its core event-driven scaling, KEDA offers the HTTP Add-on, which specifically targets HTTP workloads. KEDA's HTTP Add-on enables dynamic scaling of HTTP servers, including from zero, based on real-time HTTP traffic patterns by integrating with the cluster's routing system, thus supporting variable workloads while maintaining event-driven scaling capabilities [34].

2) Node-level autoscaling

Node-level autoscaling in Kubernetes involves dynamically adding or removing nodes from a cluster to match workload demands [8]. Unlike pod-level autoscaling, which adjusts the number of replicas or resource requests within existing nodes, node-level autoscaling focuses on scaling the underlying infrastructure, ensuring that there are sufficient worker nodes to accommodate unschedulable pods [8]. This approach optimizes resource utilization by maintaining an optimal number of nodes, preventing both over-provisioning and under-provisioning of cluster resources.

Provisioning in Kubernetes Node Autoscaling refers to dynamically adding nodes to the cluster when existing resources are insufficient to accommodate scheduled pods [8]. Consolidation optimizes resource usage and reduces costs by rescheduling workloads onto fewer nodes, allowing underutilized nodes to be terminated without impacting application availability [8]. Cluster Autoscaler and Karpenter are the two primary node autoscalers in Kubernetes, currently sponsored by SIG Autoscaling [8].

a) Cluster Autoscaler

Cluster Autoscaler (CA) is a standalone program that dynamically adjusts the size of a Kubernetes cluster by scaling nodes up or down based on pod scheduling needs [35]. CA supports a wide range of cloud providers, including AWS, Azure, GCE (Google Cloud Engine), Alibaba Cloud, Huawei Cloud, DigitalOcean, Hetzner, Scaleway, and others, offering extensive compatibility across diverse cloud environments [35]. It interacts with cloud provider-specific APIs for node provisioning and termination, allowing seamless integration and dynamic scaling [35]. Cluster Autoscaler also considers several constraints that can prevent node removal, such as restrictive PodDisruptionBudgets (PDBs), kube-system pods without PDBs, pods using local storage, and pods with strict scheduling requirements like node selectors or node affinity rules [35]. It simulates kube-scheduler behavior to ensure that all pods can be safely rescheduled on other nodes before initiating node termination [35].

b) Karpenter

Karpenter is a modern K8s autoscaling tool aimed at cloud environments, first and foremost at AWS with later added support other cloud providers such as Azure, AlibabaCloud,

1 GCP, Proxmox [36] [37]. It supports mixed-architecture clusters, including ARM and AMD type processors, and can operate across multiple availability zones, ensuring high availability and fault tolerance [36]. Additionally, it supports custom operating system images and NodePools configuration, providing flexibility for advanced deployment scenarios [36].

3) Cluster-wide autoscaling

Cluster-wide autoscaling in Kubernetes is designed to automatically adjust the resources required for critical cluster components based on overall cluster size and workload demands [7]. Unlike pod-level or node-level autoscaling, cluster-wide autoscaling focuses on ensuring that critical services maintain optimal performance as the cluster grows or shrinks. It dynamically adjusts resources like replicas and memory allocations for these services to ensure consistent performance and availability. This approach is crucial for maintaining high availability and efficient resource utilization in large-scale Kubernetes environments. A prominent example of cluster-wide autoscaling is DNS Horizontal Autoscaling, which adjusts the number of CoreDNS replicas based on the size of the cluster [38]. Since CoreDNS handles DNS-based service discovery within Kubernetes clusters, its performance is closely linked to the number of nodes and pods [39].

a) Cluster Proportional Autoscaler (CPA)

The Cluster Proportional Autoscaler (CPA) is a lightweight autoscaler in Kubernetes designed to adjust the number of replicas for critical cluster components proportionally to the size of the cluster [13]. The Cluster Proportional Autoscaler (CPA) provides 2 flexible control patterns to manage the scaling behavior of critical system components: Linear Mode and Ladder Mode [13].

Linear Mode scales replicas proportionally to the cluster size using a straightforward linear relationship [13]. It is suitable for components where demand grows consistently with the number of nodes or CPU cores.

$$\text{Replicas} = \max \left(\left\lceil \text{Cores} \times \frac{1}{\text{Cores Per Replica}} \right\rceil, \left\lceil \text{Nodes} \times \frac{1}{\text{Nodes Per Replica}} \right\rceil \right)$$

Figure 7: Linear Mode scaling formula

In Ladder Mode, replicas are adjusted according to predefined steps that correspond to specific node or core count range [13]. This approach avoids rapid fluctuations by scaling in controlled increments. Ladder Mode works by mapping the number of replicas to specific node or core count ranges. Each range corresponds to a fixed number of replicas, and when the cluster size enters a new range, the number of replicas is adjusted accordingly. For example, in a scenario where a cluster's size is measured by the number of nodes, the configuration might be set as follows:

- 1 to 5 nodes → 2 replicas
- 6 to 10 nodes → 4 replicas
- 11 to 20 nodes → 6 replicas
- 21 to 30 nodes → 8 replicas

In this case, if the cluster grows from 9 to 11 nodes, Ladder Mode increases the replicas from 4 to 6. Conversely, if the cluster size decreases to 5 nodes, replicas are reduced from 4 to 2.

```
data:
  ladder: |-
    {
      "coresToReplicas":
      [
        [ 1, 1 ],
        [ 64, 3 ],
        [ 512, 5 ],
        [ 1024, 7 ],
        [ 2048, 10 ],
        [ 4096, 15 ]
      ],
      "nodesToReplicas":
      [
        [ 1, 1 ],
        [ 2, 2 ]
      ]
    }
```

Figure 8: Example ConfigMap of Ladder

b) Cluster Proportional Vertical Autoscaler

The Cluster Proportional Vertical Autoscaler (CPVA) is an advanced autoscaler in Kubernetes designed to adjust resource requests and limits (CPU and memory) for critical system components proportionally to the overall cluster size [14]. Unlike traditional vertical pod autoscalers that focus solely on individual pod metrics, CPVA takes a more holistic approach by considering the entire cluster's capacity and adjusting resources accordingly. It is implemented using a Kubernetes Golang API client that connects to the Kubernetes API server using default service account credentials, enabling it to run inside pods and dynamically adjust resource requests and limits for critical system components [14].

CPVA calculates resource requests and limits based on the following parameters [14]:

- **Base:** The baseline quantity required.
- **Max:** The maximum allowed quantity.
- **Step:** The incremental number of resources to add.
- **CoresPerStep:** The number of cores required to trigger an increase.
- **NodesPerStep:** The number of nodes required to trigger an increase.

III. RESEARCH CHALLENGES AND ISSUES

Kubernetes, despite being the leading platform for container orchestration, faces several challenges and issues in scaling, which can significantly impact its efficiency and reliability in diverse environment.

A. Security Challenges

Scaling in Kubernetes introduces several security challenges that can be exploited by malicious actors to compromise system performance, resource utilization, and cost efficiency. Kubernetes' autoscaling mechanisms,

particularly the Horizontal Pod Autoscaler (HPA) and Cluster Autoscaler, can be exploited to launch Economic Denial of Sustainability (EDoS) attacks [40]. In these attacks, adversaries generate artificial demand, causing the autoscaling mechanism to continuously provision additional resources [40]. This process leads to inflated cloud costs and unsustainable financial pressure on the organization, as the victim is forced to pay for unnecessary computing power and infrastructure that only serve to handle the attacker's fabricated load. YoYo attacks are a variant of burst attacks where adversaries intermittently flood the system with traffic, causing the autoscaling mechanism to oscillate between scaling up and down [40] [41]. This fluctuation not only degrades performance but also incurs financial costs due to the repeated provisioning and de-provisioning of resources.

B. Stateful Workload Scaling

Scaling stateful applications in Kubernetes presents significant challenges due to their dependency on persistent storage and the necessity for maintaining data consistency across distributed systems. StatefulSets in Kubernetes are designed to manage stateful applications that require stable network identities and persistent storage [42]. Unlike Deployments, StatefulSets ensure that each pod has a unique, consistent identity and persistent volume that persists even if the pod is rescheduled [42]. This makes them ideal for databases and distributed systems requiring ordered startup, scaling, and termination. StatefulSets in Kubernetes face significant challenges with Cluster Autoscaler (CA) due to their dependency on Persistent Volumes (PVs) and strict zone constraints [35]. When a StatefulSet pod is pending, CA does not scale up if the required PV is already bound to a specific zone that has reached its scaling limit [35]. This is because CA cannot influence the zone selection for PVs, leading to a situation where nodes cannot be added even if other zones have capacity [35]. This causes scheduling failures and resource underutilization.

C. Complexity and reactive nature

Kubernetes provides multiple autoscaling mechanisms, including the Horizontal Pod Autoscaler (HPA), Vertical Pod Autoscaler (VPA), Cluster Autoscaler (CA), and custom metrics-based autoscaling solutions. However, configuring these autoscalers effectively presents several challenges. When multiple autoscalers are deployed simultaneously, conflicts can arise in scaling decisions, leading to inconsistent resource allocation and potential performance degradation [43]. A significant issue is the reliance on metrics from tools such as Metrics Server and Prometheus, which may not provide accurate or timely data, leading to suboptimal scaling decisions. Moreover, reactive autoscalers like HPA adjust resources after workload changes have occurred, introducing latency in scaling actions [44]. This reactive approach fails to anticipate workload spikes, resulting in performance bottlenecks and resource shortages during high-demand periods.

D. Resource Allocation and Fragmentation

Efficient resource allocation is a significant challenge in dynamic scaling scenarios. Kubernetes uses a bin-packing algorithm to schedule pods on nodes [45]. It can lead to resource fragmentation as pods with varying resource

requirements are scheduled and rescheduled. This results in nodes having fragmented resources that are insufficient to schedule new pods, leading to inefficient utilization and increased scheduling complexities. Furthermore, developers often overestimate resource requests and limits to avoid pod evictions, leading to underutilization and inflated infrastructure costs. The inefficiencies in resource allocation are exacerbated in multi-tenant environments where workloads have diverse and unpredictable resource demands.

IV. DIRECTIONS FOR FUTURE RESEARCH

Despite the advancements in Kubernetes scaling mechanisms, there remain significant challenges and opportunities for future research. As containerized applications become increasingly complex and distributed, scaling solutions must evolve to ensure efficiency, reliability, and cost-effectiveness.

A. AI-Driven Observability and Self-Healing Clusters

AI-driven observability frameworks are emerging as a solution to the limitations of traditional monitoring tools, which often suffer from latency in detecting anomalies and are limited by static alert thresholds. Future research should focus on enhancing AI-powered observability with real-time anomaly detection, root cause analysis, and predictive maintenance. The integration of Prometheus and Grafana with AI models for predictive analytics, as demonstrated in the study on AI-driven observability for Kubernetes clusters, reduced Mean Time to Resolution (MTTR) and optimized resource allocation [44]. Future work could investigate the application of unsupervised learning models for anomaly detection, causality analysis, and automated remediation to enable self-healing Kubernetes clusters.

B. Scaling Stateful Applications and Persistent Volume Autoscaling

Scaling stateful applications remain a challenge due to the complexities of maintaining data consistency and persistent storage. Traditional Kubernetes autoscalers like HPA and VPA do not support volume scaling, leading to resource wastage or service disruptions. The development of Persistent Volume Autoscaler (PVA) offers a promising approach by dynamically scaling volume resources based on utilization metrics [46]. Further research is needed to enhance the resilience of stateful workloads by investigating distributed storage systems and hybrid autoscalers that jointly optimize compute and storage scaling.

C. Hybrid and Multi-Cloud Scaling Strategies

With the increasing adoption of multi-cloud and hybrid cloud environments, Kubernetes clusters are becoming more distributed, requiring advanced scaling strategies to optimize resource utilization across heterogeneous infrastructures. Cluster Autoscaler (CA-NAP) demonstrated the potential of provisioning nodes from multiple pools with varied sizes, enhancing resource matching and reducing cost [47]. Future research should explore hybrid autoscalers that intelligently distribute workloads across on-premises, public, and edge cloud resources. This involves developing unified scaling policies that account for cross-cloud network latency, data locality, and cost optimization while ensuring application consistency.

D. Integration of Autoscaling Mechanisms

Kubernetes provides multiple autoscalers (HPA, VPA, Cluster Autoscaler, KEDA), but conflicts can arise when these mechanisms are deployed together, leading to inconsistent scaling decisions. Future research should focus on developing unified autoscaling frameworks that orchestrate different autoscalers harmoniously. This involves creating coordination layers that prioritize scaling decisions and resolve conflicts between horizontal, vertical, and event-driven autoscalers. Investigating policy-driven autoscaling frameworks that adapt to application-specific requirements and workload patterns will be crucial for achieving consistent and reliable scaling.

E. Predictive and Proactive Autoscaling Algorithms

Traditional autoscalers in Kubernetes, such as HPA and VPA, primarily rely on reactive scaling triggered by static thresholds or historical metrics. This reactive nature can lead to scaling delays and resource inefficiencies. Future research should explore predictive autoscaling algorithms that leverage advanced machine learning techniques for workload forecasting. For example, AHPA on Alibaba Cloud demonstrated the potential of proactive prediction using decomposition-based statistical methods to accurately anticipate workload fluctuations and adjust pod counts accordingly [48]. Similarly, integrating KEDA with custom machine learning models for dynamic pod management showed promise in reducing scaling latency and improving resource utilization [32]. Research should further investigate the integration of time-series forecasting models (e.g., ARIMA, LSTM) and reinforcement learning to enable anticipatory scaling decisions.

V. CONCLUSION

This comprehensive review has thoroughly examined the scalability mechanisms in Kubernetes, highlighting the diverse strategies employed at both the application and cluster levels. At the application level, Kubernetes utilizes the Horizontal Pod Autoscaler (HPA) and Vertical Pod Autoscaler (VPA) to dynamically adjust pod replicas and resource allocations based on real-time metrics, ensuring efficient resource utilization and workload management. Event-driven scaling frameworks like KEDA extend these capabilities by enabling reactive autoscaling based on custom external metrics, which is particularly effective in event-driven and microservices architectures. At the cluster level, the Cluster Autoscaler (CA) and Karpenter enhance node provisioning by dynamically adding or removing nodes to align with workload demands. Additionally, the Cluster Proportional Autoscaler (CPA) and Cluster Proportional Vertical Autoscaler (CPVA) optimize resource allocation by proportionally adjusting critical cluster components and resource requests based on overall cluster size, thus maintaining consistent performance and availability. Despite these advancements, challenges such as reactive scaling delays, resource fragmentation, and conflicts among multiple autoscalers continue to hinder optimal scalability. Moreover, scaling stateful applications remains complex due to persistent volume constraints and data consistency requirements, necessitating specialized solutions like the Persistent Volume Autoscaler (PVA).

While Kubernetes has made significant strides in automated scaling, this review underscores the need for more intelligent, proactive, and unified autoscaling solutions to address existing challenges. Security vulnerabilities, including Economic Denial of Sustainability (EDoS) attacks that exploit autoscaler behaviors, further complicate scaling strategies. Emerging trends in AI-driven observability and predictive analytics provide promising avenues for proactive autoscaling, anomaly detection, and self-healing clusters. The growing adoption of hybrid and multi-cloud environments necessitates unified scaling frameworks that harmonize autoscaling decisions across distributed infrastructures. By advancing these areas and leveraging next-generation AI and predictive technologies, Kubernetes is poised to remain the leading platform for scalable, resilient, and cost-effective cloud-native applications.

VI. ACKNOWLEDGMENTS

The researchers acknowledge the great support and assistance granted to them by the academic staff of SLIIT. Their dedication makes the faculty to constantly maintain their academic and research standard by creating a comfortable learning culture.

VII. REFERENCES

- [1] "Application Container Market Size & Share Analysis - Growth Trends & Forecasts (2024 - 2029) Source: <https://www.mordorintelligence.com/industry-reports/application-container-market>," 2024.
- [2] B. A. Christudas, "Java Microservices and Containers in the Cloud: With Spring Boot, Kafka, PostgreSQL, Kubernetes, Helm, Terraform and AWS EKS," 2024, p. 345-404.
- [3] T. Eddy, V. L. Dimitri, P. Davy, L. Bert and J. Wouter, "A Comprehensive Feature Comparison Study of Open-Source Container Orchestration Frameworks," 2020.
- [4] Amazon Web Services, "What is Container Orchestration?," [Online]. Available: <https://aws.amazon.com/what-is/container-orchestration/>. [Accessed 17 February 2025].
- [5] The Linux Foundation, "Kubernetes Project Journey Report," 2023. [Online]. Available: <https://www.cncf.io/reports/kubernetes-project-journey-report/>. [Accessed 18 February 2025].
- [6] Z. Ren and J. Xu, "Research on the implementation of Kubernetes automatic scaling," *Second International Conference on Electronic Information Engineering, Big Data, and Computer Technology (EIBDCT 2023)*, vol. 12642, 2023.
- [7] The Linux Foundation, "Autoscaling Workloads," [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/autoscaling/>. [Accessed 18 February 2025].
- [8] The Linux Foundation, "Node Autoscaling," [Online]. Available: <https://kubernetes.io/docs/concepts/cluster-administration/node-autoscaling/>. [Accessed 18 February 2025].
- [9] The Linux Foundation, "Horizontal Pod Autoscaling," [Online]. Available: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. [Accessed 18 February 2025].
- [10] "vertical-pod-autoscaler," [Online]. Available: <https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler>. [Accessed 18 February 2025].
- [11] S. Kavuri, "Integrating Kubernetes Autoscaling for Cost Efficiency in Cloud Services," *International Journal of Scientific Research in Computer Science, Engineering*, vol. 10, no. 5, pp. 480-502, 2024.
- [12] R. Williamson, "Autoscaling consumers in event driven architectures," 29 May 2024. [Online]. Available: <https://www.cncf.io/blog/2024/05/29/autoscaling-consumers-in-event-driven-architectures/>. [Accessed 18 February 2025].

- [13] "cluster-proportional-autoscaler," [Online]. Available: <https://github.com/kubernetes-sigs/cluster-proportional-autoscaler>. [Accessed 19 February 2025].
- [14] "cluster-proportional-vertical-autoscaler (BETA)," [Online]. Available: <https://github.com/kubernetes-sigs/cluster-proportional-vertical-autoscaler>. [Accessed 18 February 2025].
- [15] Amazon Web Services, "Cluster Autoscaler," [Online]. Available: <https://docs.aws.amazon.com/eks/latest/best-practices/cas.html>. [Accessed 18 February 2025].
- [16] "Concepts Understand key concepts of Karpenter," [Online]. Available: <https://karpenter.sh/docs/concepts/#kubernetes-cluster-autoscaler>. [Accessed 18 February 2025].
- [17] "FreeBSD Containers and Orchestration," [Online]. Available: <https://wiki.freebsd.org/Containers>. [Accessed 19 February 2025].
- [18] P. Daniel and T. Andrew, "Solaris Zones: Operating System Support for Consolidating Commercial Workloads," in *Proceedings of LISA '04: Eighteenth Systems Administration Conference*, Atlanta, 2004.
- [19] Sun Microsystems, Inc, "Solaris 10 What's New," 2004.
- [20] B. Brendan, G. Brian, O. David, B. Eric and W. John, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50-57, 2016.
- [21] V. Abhishek, P. Luis, K. Madhukar, O. David, T. Eric and W. John, "Large-scale cluster management at Google with Borg," in *EuroSys '15: Proceedings of the Tenth European Conference on Computer Systems*, New York, USA, 2015.
- [22] S. Malte, K. Andy, A.-E.-M. Michael and W. John, "Omega: flexible, scalable schedulers for large compute clusters," in *EuroSys '13: Proceedings of the 8th ACM European Conference on Computer Systems*, Prague, Czech Republic, 2013.
- [23] Google inc, "2014 Year in Review: Making open source #1 through Kubernetes," 31 December 2014. [Online]. Available: <https://cloudplatform.googleblog.com/2014/12/2014-year-in-review-Making-open-source-1-through-Kubernetes-and-Google-Container-Engine.html>. [Accessed 19 February 2025].
- [24] The Linux Foundation, "Cluster Architecture," [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/>. [Accessed 19 February 2025].
- [25] The Linux Foundation, "Pods," [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/pods/>. [Accessed 19 February 2025].
- [26] The Linux Foundation, "HorizontalPodAutoscaler Walkthrough," [Online]. Available: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>. [Accessed 19 February 2025].
- [27] The Linux Foundation, "Kubernetes API," [Online]. Available: <https://kubernetes.io/docs/reference/kubernetes-api/>. [Accessed 20 February 2025].
- [28] The Linux Foundation, "HorizontalPodAutoscaler," [Online]. Available: <https://kubernetes.io/docs/reference/kubernetes-api/workload-resources/horizontal-pod-autoscaler-v1/>. [Accessed 20 February 2025].
- [29] The Linux Foundation, "HorizontalPodAutoscaler," [Online]. Available: <https://kubernetes.io/docs/reference/kubernetes-api/workload-resources/horizontal-pod-autoscaler-v2/>. [Accessed 20 February 2025].
- [30] N. Thanh-Tung, Y. Yu-Jin, K. Tachong, P. Dae-Heon and K. Sehan, "Horizontal Pod Autoscaling in Kubernetes for Elastic Container Orchestration," vol. 20, no. 16, 2020.
- [31] "KEDA," [Online]. Available: <https://keda.sh/>. [Accessed 20 February 2025].
- [32] L. N. C. J. Prashanth, "From Reactive to Predictive: Harnessing KEDA and Custom Machine Learning for Dynamic POD Management," *International Journal of Artificial Intelligence & Machine Learning (IJAIML)*, vol. 3, no. 2, pp. 14-39, 2024.
- [33] [Online]. Available: <https://github.com/kedacore/keda/tree/main/config/crd/bases>. [Accessed 20 February 2025].
- [34] [Online]. Available: <https://github.com/kedacore/http-add-on>. [Accessed 25 February 2025].
- [35] "cluster-autoscaler," [Online]. Available: <https://github.com/kubernetes/autoscaler/tree/master/cluster-autoscaler>. [Accessed 20 February 2025].
- [36] [Online]. Available: <https://karpenter.sh/>. [Accessed 21 February 2025].
- [37] [Online]. Available: <https://github.com/kubernetes-sigs/karpenter>. [Accessed 21 February 2025].
- [38] The Linux Foundation, "Autoscale the DNS Service in a Cluster," [Online]. Available: <https://kubernetes.io/docs/tasks/administer-cluster/dns-horizontal-autoscaling/>. [Accessed 21 February 2025].
- [39] "Using CoreDNS for Service Discovery," [Online]. Available: <https://kubernetes.io/docs/tasks/administer-cluster/coredns/>. [Accessed 21 February 2025].
- [40] B. D. Ronen and B.-B. Anat, "Kubernetes Autoscaling: YoYo Attack Vulnerability and Mitigation," in *International Conference on Cloud Computing and Services Science*, 2021.
- [41] R. Vani, S. Muzafer, K. Darjan, S. Dragisa, H. Amor, A. Melisa and T. Srivashan, "Security-Enhanced QoS-Aware Autoscaling of Kubernetes Pods Using," *Journal of Intelligent Management Decision*, vol. 3, no. 3, 2024.
- [42] The Linux Foundation, "StatefulSets," [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/controllers/statefulsets/>. [Accessed 22 February 2025].
- [43] T. Minh-Ngoc, V. Dinh-Dai and K. Younghun, "A Survey of Autoscaling in Kubernetes," in *Thirteenth International Conference on Ubiquitous and Future Networks (ICUFN)*, Barcelona, Spain, 2022.
- [44] P. Sandeep, N. Sai Rama Krishna and K. Srikanth Reddy, "Scaling Kubernetes Clusters with AI-Driven Observability for Improved Service Reliability," *Journal of AI-Powered Medical Innovations*, vol. 3, no. 1, pp. 39-52, 2024.
- [45] The Linux Foundation, "Resource Bin Packing," [Online]. Available: <https://kubernetes.io/docs/concepts/scheduling-eviction/resource-bin-packing/>. [Accessed 22 February 2025].
- [46] N. Ji-Hyun, Y. Hyeon-Jin, K. Hyeongbin, K. Heeju, L. Hee-Dong, S. Jae-Hyuck and N. Seo-Young, "PVA: The Persistent Volume Autoscaler for Stateful Applications in Kubernetes," *IEEE Access*, vol. 12, pp. 179130-179143, 2024.
- [47] A. T. Mulugeta, T. Johan, E. Erik and P. Guillaume, "An Experimental Evaluation of the Kubernetes Cluster Autoscaler in the Cloud," in *12th IEEE International Conference on Cloud Computing Technology and Science*, Bangkok, Thailand, 2020.
- [48] Z. Zhiqiang, Z. Chaoli, M. Lingna, G. Jing, Q. Huajie, W. Qingsong, S. Liang, L. Peng and T. Zhimin, "AHPA: Adaptive Horizontal Pod Autoscaling Systems on Alibaba Cloud," in *The Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI-23)*, 2023.